

ASENA: Self-evolving Agents for Embodied Navigation

An-Chieh Cheng¹, Isabella Liu¹, Edmund Bu², Johan Bjorck¹, Hongxu Yin¹, Zhengyi Luo¹, Jan Kautz¹, Linxi “Jim” Fan¹, Yuke Zhu¹ and Sifei Liu¹

¹NVIDIA, ²University of California, San Diego

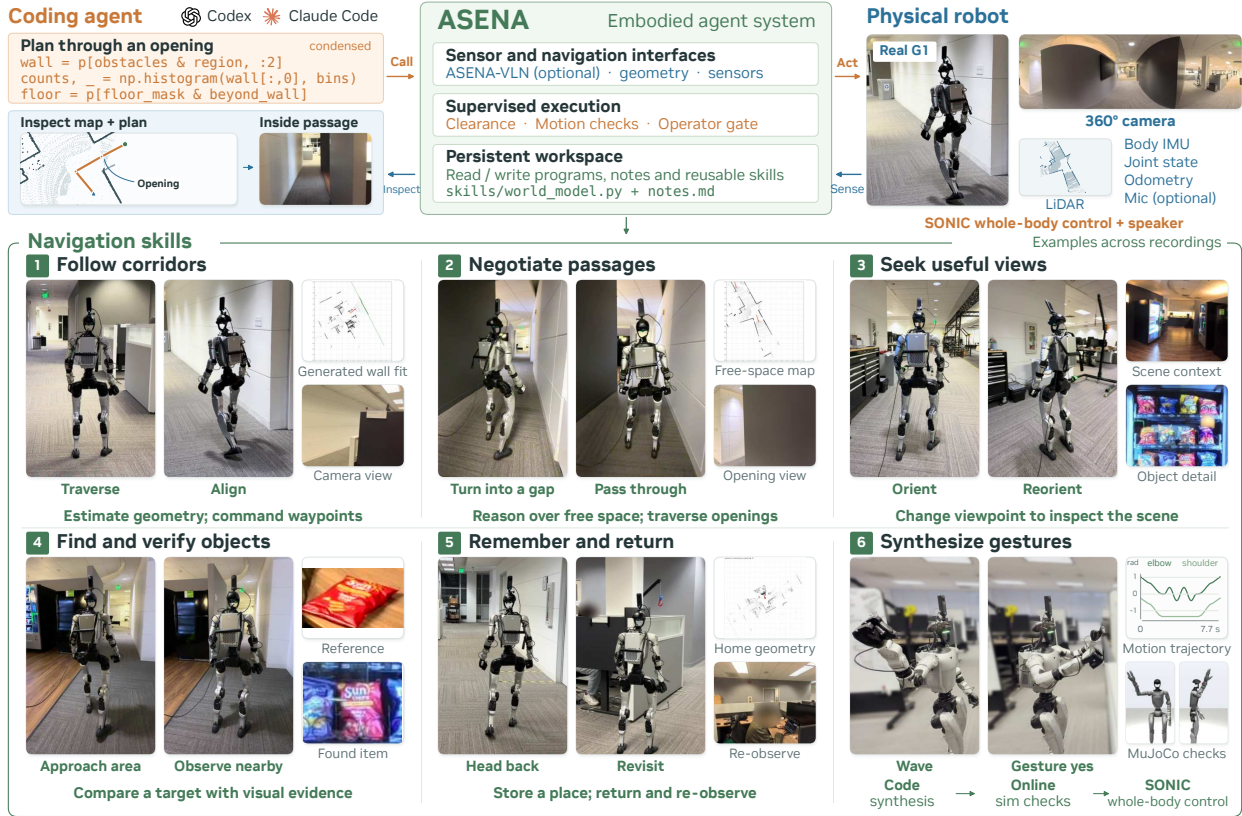


Figure 1: ASENA connects coding agents to physical robot skills. Programs combine sensing, computation, supervised execution, and reusable skills, with learned navigation as an optional tool. The coding example analyzes LiDAR geometry to propose a route through an opening. The inset shows the subsequent observation inside the passage. The bottom panels show a subset of skills evolved by the system. <https://asena-bot.github.io>.

Abstract

We present ASENA, an embodied agent system that connects general-purpose coding agents to robot sensing, computation, supervised execution, and persistent experience. Agents can write and execute programs, inspect recorded outcomes, repair failures, and reuse notes and executable skills while keeping their model weights fixed. We further introduce ASENA-VLN, a 4B monocular navigation policy that serves as an optional tool within this programmable system. ASENA-VLN predicts body-frame trajectories for both extended routes and short-horizon behaviors using a shared vision-language decoder trained on route instructions, visual question answering, and a newly curated dataset of geometry-derived atomic navigation tasks. As a standalone policy, ASENA-VLN achieves state-of-the-art success rates of 68.7% on R2R and 70.2% on RxR. When integrated with a coding agent, learned navigation improves ASENA’s success rate by 11 percentage points on both agentic benchmarks while reducing execution time. Through persistent workspace evolution and simulator feedback, ten passes over recurring 100-task subsets further improve success from 72% to 98% on R2R and from 65% to 89% on RxR. On embodied

question answering, ASENS achieves state-of-the-art accuracy with fewer interaction steps. Finally, real-world demonstrations on a Unitree G1 combine search, visual inspection, spatial reasoning, and synthesized gestures without a pre-built map, illustrating how online programming extends robot behavior beyond route following and predefined skills.

1. Introduction

A useful navigation robot must do more than reach a destination. If asked to check whether a vending machine contains an item and return with an answer, it must decide where to search, recognize the item, remember the user’s location, and communicate what it found. Navigation is therefore only one part of a larger task whose next step depends on what the robot observes.

A learned navigation policy provides movement capabilities, but such requests also require deciding what to inspect, compute, and communicate. These procedures depend on new observations and may extend beyond predefined skills. Coding agents can construct and revise them during a mission through programmatic access to sensing, computation, simulation, and action. Physical deployment also requires feedback tied to outcomes, safeguards that remain effective as programs change, and a persistent workspace for retaining useful procedures.

We present ASENS, an embodied agent system providing these capabilities (Fig. 1). The agent can read sensors, derive geometry, execute code, inspect generated visualizations, and request robot actions. ASENS connects these operations through shared interfaces, supervised execution, and linked execution records. A persistent workspace stores notes and executable skills, allowing the system to improve through program revision while keeping its model weights fixed.

Learned policies can further expand this programmable toolset when suitable training data are available. Delegating motion to a specialized policy reduces repeated agent decisions and supports navigation behaviors that are difficult to specify geometrically. We develop ASENS-VLN, a 4B monocular navigation policy trained on route instructions and geometry-derived navigation atoms. Building on vision-and-language navigation (VLN) [1–5], ASENS-VLN predicts body-frame trajectories for both extended routes and short-horizon goals, such as approaching an object or passing through a doorway. We curate a new atomic navigation dataset that directly supervises these reusable, fine-grained behaviors and complements conventional route-level training. The policy remains an optional tool: a program may call it directly, combine it with geometric navigation, or complete a task using other available operations.

Our contributions are: (1) an embodied agent system for inspectable, supervised task programming and persistent skill acquisition with coding agents; (2) a state-of-the-art monocular navigation policy that supports both route following and reusable atomic skills; and (3) evaluations showing that fixed-weight workspace evolution improves task performance. We evaluate navigation and embodied question answering (EQA) in simulation and demonstrate complex instruction following and user interaction on a real-world Unitree G1 robot operating without a pre-built map.

2. Related Work

Code as policies. Code as Policies [6] composes perception and control APIs in programs. CaP-X [7] provides CaP-Gym and benchmarks how API abstraction and execution feedback affect coding agents for manipulation. ASPIRE [8] uses multimodal execution traces to diagnose failures, repair robot programs, and retain reusable skills.

Agentic self-improvement. Voyager [9] retains and repairs executable skills. Reflexion [10] retains textual feedback without weight updates. ENPIRE [11] couples real-robot reset and verification with agent-driven program synthesis and policy training. In contrast, ASENS studies how a persistent workspace can evolve

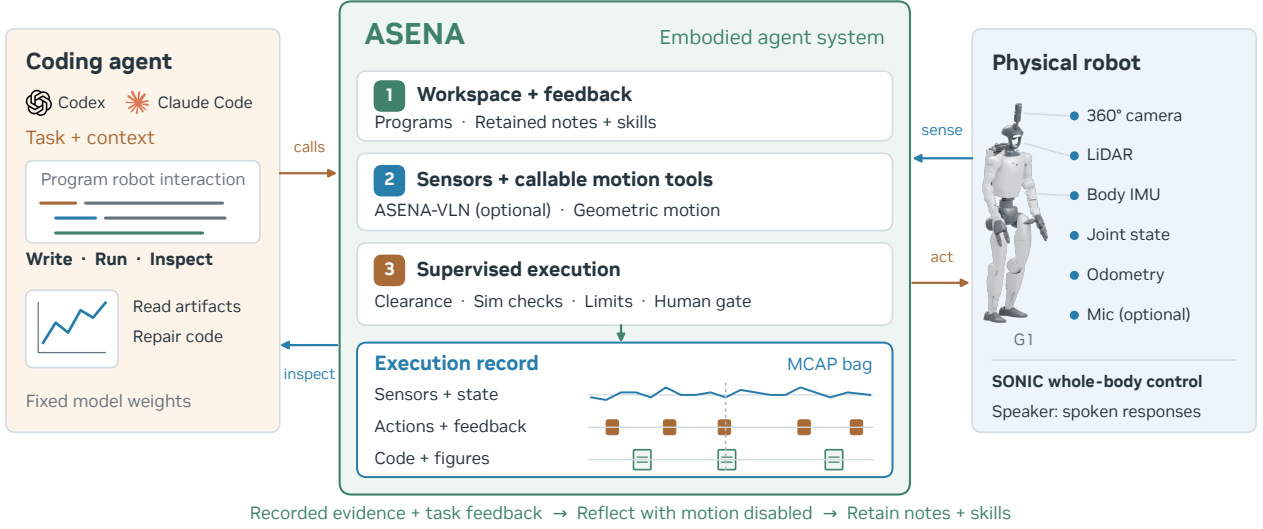


Figure 2: ASENSA connects coding agents to physical robots. The system integrates robot sensing, a persistent workspace, supervised execution, and an optional learned navigation policy. Each run is stored as an MCAP record [24] that aligns sensor and robot states with actions, operator feedback, code, and visualizations for inspection and replay. The agent reflects on these records to update its notes and skills while robot motion is disabled.

around a pretrained navigation policy while all model weights remain fixed. We quantify this workspace evolution in simulation and demonstrate the resulting supervised capabilities on a physical robot.

Language-guided navigation. Topological approaches such as DUET and ETPNav [12; 13] and video-to-action policies such as NaVid, NaVILA, and StreamVLN [3–5] address visual instruction following. LM-Nav [14] composes pretrained models, while InstructNav [15] combines zero-shot reasoning with generic motion primitives. DualVLN and Qwen-RobotNav [16; 17] integrate high-level reasoning with learned navigation policies, and Uni-LaViRA [18] studies unified language–vision–action translation. Our VLN policy is most closely related to video-to-action navigation policies, but focuses on a lightweight 4B model with fine-grained atomic capabilities that coding agents can compose with other tools and retained programs. We evaluate both its standalone navigation performance and its utility within the full embodied agent system.

Embodied question answering. Explore-EQA [19] studies active exploration until an agent can answer with sufficient confidence, while 3D-Mem and GraphEQA [20; 21] organize observations into spatial memories. MemoryEQA [22] extends the setting to longer-horizon, multi-target questions, and FAST-EQA [23] focuses on efficient evidence gathering. We use these EQA settings to evaluate ASENSA’s ability to compose perception, navigation, and memory tools, and to improve through revisions to persistent notes and executable skills while keeping model weights fixed.

3. Method

ASENSA gives a coding agent programmatic access to a physical robot (Fig. 2). It brings sensing, computation, visualization, simulation, and action into a unified write–run–inspect workflow. Each run is subject to physical safeguards and recorded for later inspection, repair, and reuse. The agent can also invoke a learned navigation policy as an optional tool within the same interface.

3.1. Grounding Programs in Robot Feedback

A program must test its assumptions against physical feedback. On G1, ASENSA exposes a 360-degree camera, LiDAR, odometry, a body IMU, and joint states. The coding agent processes and visualizes these inputs to

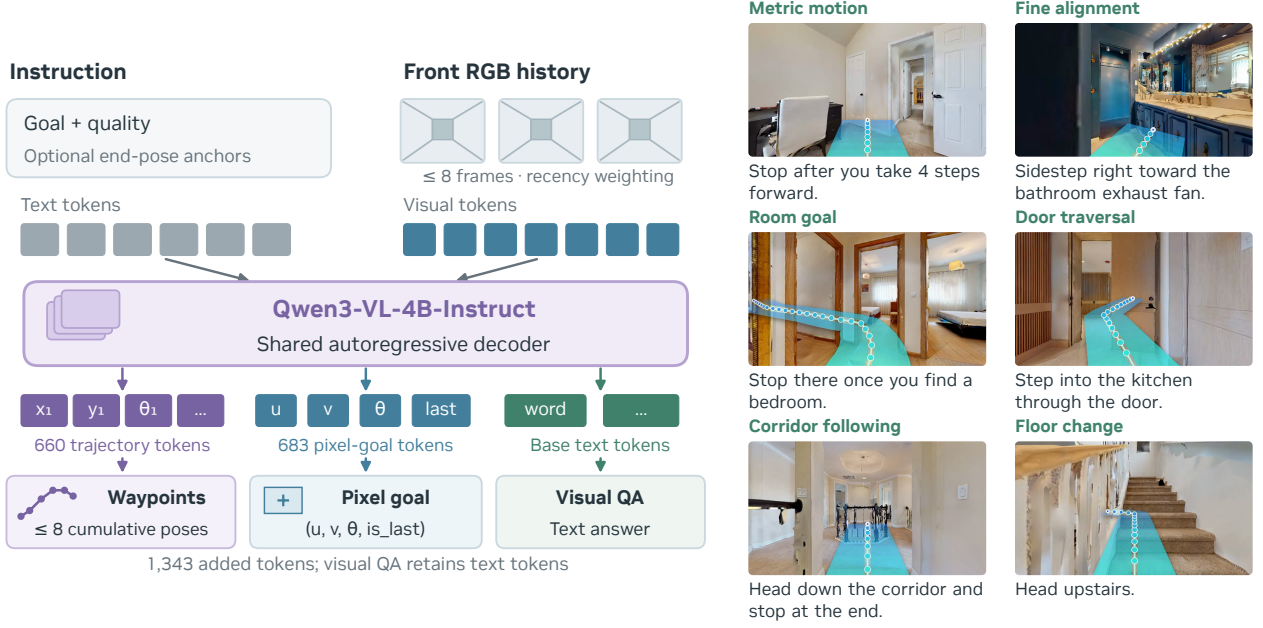


Figure 3: Navigation policy architecture and atomic training examples. Left: language and monocular history enter a shared autoregressive decoder. Separate token blocks encode trajectories and pixel goals; visual QA uses text tokens. Motion tokens decode to body-frame waypoints. Architecture glyphs are schematic. Right: recorded training observations with instructions and overlaid trajectories. The overlays do not use metric camera projection. More examples appear in Fig. 6.

estimate geometry, inspect objects, and monitor execution. When an action differs from its intent, the agent can inspect the recorded evidence and revise its program.

All physical actions requested by generated programs pass through ASENA’s execution interface and are carried out by the SONIC whole-body controller [25], subject to the validation and approval mechanisms described below. Within this interface, the agent can use geometric navigation tools for explicit waypoint and clearance reasoning or invoke the learned policy for language-conditioned motion.

ASENA wraps sensing, control, and safety interfaces in callable atomic skills, such as approaching an object, passing through a doorway, speaking, or gesturing. The agent combines these operations with computation and interaction logic to create larger reusable skills, adapting task-level programs while retaining the same physical execution interfaces.

Safeguards. Compared with tabletop manipulation, whole-body humanoid motion introduces broader safety risks: the robot must maintain balance while moving its full mass through a shared physical space. Because synthesized programs can change this behavior quickly, we employ safeguards that control physical execution independently of program generation. We first validate synthesized gestures online in MuJoCo, checking pose continuity, joint and motion limits, and sampled self-collision. When a proposal fails validation, we return the violated constraint to the agent for repair. We further require operator approval before any validated motion is executed by the SONIC [25].

For navigation, we use LiDAR-based occupancy maps for route planning and obstacle-clearance checks. We additionally employ an independent safeguard that enforces motion limits and rejects commands when odometry is stale or the battery level is low. Heartbeat-triggered stops and manual emergency halts provide further protection during execution. Together, these mechanisms provide layered safeguards across program synthesis, simulation-based validation, human authorization, and physical execution.

Table 1: Standalone monocular VLN-CE policy results on R2R and RxR val_unseen. ASENA-VLN-4B achieves a state-of-the-art result compared to prior work, with significantly lower navigation error and higher nDTW.

Method	R2R				RxR			
	NE ↓	OSR ↑	SR ↑	SPL ↑	NE ↓	nDTW ↑	SR ↑	SPL ↑
NaVid [3]	5.72	49.2	41.9	36.5	5.72	–	45.7	38.2
Uni-NaVid [26]	5.58	53.3	47.0	42.7	6.24	–	48.7	40.9
NaVILA [4]	5.22	62.5	54.0	49.0	6.77	58.8	49.3	44.0
StreamVLN [5]	4.98	64.2	56.9	51.9	6.22	61.9	52.9	46.0
NaVIDA [27]	4.32	69.5	61.4	54.7	5.23	67.0	57.4	49.6
JanusVLN [28]	4.78	65.2	60.5	56.8	6.06	62.1	56.2	47.5
DecoVLN [29]	5.01	63.5	56.3	50.5	5.73	63.5	54.2	46.3
ActiveVLN [30]	5.31	58.3	50.1	43.7	5.84	58.1	50.7	41.2
Qwen-VLA-Instruct [31]	5.10	69.0	57.5	51.2	5.80	57.1	59.6	47.8
InternVLA-N1 (S2, SPF) [32]	4.25	68.3	60.9	55.2	5.71	46.8	63.5	55.0
RynnBrain-Nav-8B (SPF) [33]	4.92	71.6	58.6	49.6	6.20	59.6	56.1	49.6
DualVLN [16]	4.05	70.7	64.3	58.5	4.58	70.0	61.4	51.8
InternVLA-N1 [32]	4.83	63.3	58.2	54.0	5.91	65.3	53.5	46.1
Qwen-RobotNav-4B [17]	4.22	73.6	66.9	60.5	4.15	68.6	71.3	61.5
ASENA-VLN-4B	3.52	72.6	68.7	64.2	3.90	73.1	70.2	59.7

Table 2: Object-goal navigation: paired SR/SPL (%), both ↑). [‡] denotes HM3D v1 result, where emphasis excludes these.

Method	HM3D	HM3D-OVON		
	v2 val	seen	syn.	unseen
VLFM	63.6/32.5	35.2/18.6	32.4/17.3	35.2/19.6
OpenFMNav [‡]	52.5/24.1	–	–	–
SG-Nav	49.6/25.5	–	–	–
TriHelper [‡]	56.5/25.3	–	–	–
WMNav [‡]	58.1/31.2	–	–	–
CogNav [‡]	72.5/26.2	–	–	–
Uni-NaVid [‡]	73.7/37.1	41.3/21.1	43.9/21.8	39.5/19.8
DAGRL+OD	–	38.5/21.1	39.0/21.4	37.1/19.8
MTU3D	–	55.0/23.6	45.0/14.7	40.8/12.1
NavFoM	–	40.1/27.1	45.4/32.6	45.2/31.9
ABot-N0	–	55.3/32.1	55.4/33.2	54.0/30.5
ApexNav	76.2/38.0	–	–	–
Qwen-RobotNav-4B	75.6/30.6	57.7/24.4	60.1/25.1	53.1/20.9
Qwen-RobotNav-8B	71.2/33.0	56.1/28.5	57.8/28.8	51.2/24.0
ASENA	80.8/38.1	60.5/33.5	65.8/37.9	64.9/38.4

3.2. Turning Execution into Retained Experience

ASENA records runs in MCAP format [24], aligning sensor and robot states with actions, operator feedback, code, and visualizations. Replaying this evidence lets the agent identify where execution diverged from its intent and repair the responsible program.

After repairing a failure, ASENA can retain the solution for future tasks. We represent the workspace at revision k as $W_k = (N_k, S_k)$, where N_k contains notes and S_k contains executable skills. A fixed coding model consults this workspace while solving tasks. Between revisions, a separate fixed-weight improver uses execution traces $\mathcal{T}_k$ and feedback f_k to update the workspace:

$$W_{k+1} = I_\phi(W_k, \mathcal{T}_k, f_k). \quad (1)$$

Model and controller weights remain fixed. In simulation, private workers may edit workspaces between passes, while writes are disabled during frozen evaluation. On G1, execution records and optional operator feedback support skill revision. Every revised program must still pass independent safety checks and receive operator approval before physical execution.

Table 3: Navigation before self-evolution on R2R and RxR Agentic Splits. NE is in meters and OSR/SR/SPL are percentages. Calls/ep. is mean tool calls over 100 episodes. Green percentages show reductions from the same backend without VLN.

Method / coding agent	VLN tool	R2R Agentic Split					RxR Agentic Split			
		NE ↓	OSR ↑	SR ↑	SPL ↑	Calls/ep. ↓	NE ↓	SR ↑	SPL ↑	Calls/ep. ↓
<i>Published zero-shot methods</i>										
InstructNav [15]	–	6.89	47.0	31.0	24.0	–	–	–	–	–
Open-Nav [34]	–	6.70	23.0	19.0	16.1	–	–	–	–	–
CA-Nav [35]	–	7.58	48.0	25.3	10.8	–	10.4	19.0	6.0	–
GC-VLN [36]	–	7.30	41.8	33.6	16.3	–	8.80	33.8	13.8	–
Three-Step Nav [37]	–	5.87	39.0	34.0	29.1	–	9.21	22.0	16.1	–
Uni-LaViRA [18]	–	3.66	73.7	60.7	47.7	–	6.48	51.3	34.0	–
Minimal (Claude Sonnet 5, SDK) [38]	–	5.80	61.3	51.3	37.8	–	–	–	–	–
<i>ASENA policy and fresh-workspace agents</i>										
ASENA-VLN-4B	–	3.16	82.0	78.0	70.7	–	4.54	62.0	52.9	–
ASENA (Claude Sonnet 5)	×	<u>5.23</u>	<u>70.0</u>	<u>61.0</u>	<u>34.2</u>	<u>83.32</u>	<u>5.80</u>	<u>50.0</u>	<u>31.9</u>	<u>103.83</u>
ASENA (Claude Sonnet 5)	✓	3.31	83.0	72.0	54.9	33.96 ↓59.2%	4.94	61.0	45.5	38.91 ↓62.5%
ASENA (GPT-6 Astra)	×	1.84	90.0	87.0	72.4	26.37	0.82	93.0	78.0	40.80
ASENA (GPT-6 Astra)	✓	<u>2.18</u>	<u>89.0</u>	88.0	72.8	14.00 ↓46.9%	<u>1.65</u>	<u>86.0</u>	<u>64.0</u>	19.98 ↓51.0%

3.3. Learning a Reusable Visual Navigation Policy

ASENA-VLN reduces repeated agent reasoning on challenging routes. This optional tool predicts bounded body-frame trajectories from language and recent monocular observations. The coding agent decides when to invoke it, checks progress, and combines it with other tools. Policy training expands navigation capabilities within the same programming interface.

One decoder, distinct output representations. We extend a lightweight Qwen3-VL-4B-Instruct [39] with dedicated token vocabularies for trajectories and pixel goals (Fig. 3). The model takes up to eight front-view frames together with a structured JSON instruction, making the policy easy to invoke as a callable tool. The input specifies a concise navigation command and can optionally include a desired end pose, anchor objects, or motion constraints. Recency-weighted visual tokens preserve temporal context while emphasizing the latest observation. A shared autoregressive decoder predicts up to eight cumulative body-frame waypoints, each represented as (x, y, θ) . Fixed bins along each axis convert trajectory tokens into explicit motion targets without requiring a separate continuous action head. Pixel-goal tokens provide auxiliary supervision by encouraging the model to localize its intended destination in the image, while visual question answering retains the model’s original text vocabulary. Because a pixel goal is not directly executable and requires an additional motion planner or, in simulation, a shortest-path follower, all evaluations in this paper use trajectory-token predictions rather than pixel goals.

Curated atomic-navigation data. A callable navigation policy must follow both extended routes and precise local commands. We therefore curate nine atomic navigation tasks in MP3D and HM3D. They cover behaviors such as moving a specified distance, rotating to a target heading, approaching an object from a given direction, aligning with a target, entering a room, passing through a doorway, following a corridor, and moving between floors. We generate and validate each task using geodesic distance, visibility, and scene geometry so that every instruction describes a feasible and measurable motion. Each instruction is paired with a waypoint trajectory and auxiliary pixel-goal supervision. These data directly teach the reusable, fine-grained operations that task programs can invoke and compose.

We train ASENA-VLN on a mixture of these atomic tasks, short-horizon ObjectNav, and vision-language-navigation data from R2R, RxR, ScaleVLN, and SRDF [1; 2; 40; 41]. To match our callable policy interface, we use GPT-5.5 [42] to re-caption the original instructions from these datasets into our structured JSON format, including concise navigation commands and, when applicable, end poses, anchor objects, and motion



Figure 4: Navigation workspace self-evolution. Success-related metrics (SR, OSR, and SPL) improve across evolution passes, while the cost per pass generally decreases. Access to the VLN tool consistently improves SPL, enabling the agent to reach destinations more efficiently.

constraints. We also include multi-view VQA data [43] to strengthen the model’s spatial understanding. Training draws 4.3M samples using a 3:1 mixture of trajectory prediction and spatial VQA.

4. Evaluation and Applications

We first evaluate ASENA-VLN as a standalone instruction-following policy and then evaluate the full ASENA system on object-goal navigation. We next study how the learned policy and retained workspace experience affect coding-agent performance with model weights fixed. Finally, we evaluate embodied question answering in simulation and demonstrate the system on a Unitree G1 robot.

4.1. Standalone VLN Policy

Table 1 evaluates ASENA-VLN independently of coding-agent planning on R2R’s shorter instructions and RxR’s longer, more detailed routes.

We use 0.25 m forward steps and 15-degree rotations. Navigation error (NE) is the final goal distance in meters. Oracle success rate (OSR) measures entering the goal region, while success rate (SR) requires stopping there. SPL accounts for path efficiency and nDTW for reference-route similarity. All four are percentages. SPF denotes the simulator shortest-path follower used by pixel-goal methods.

4.2. Object-Goal Navigation

Table 2 extends the evaluation from following a given route to searching for an object that may not initially be visible. Because ASENA-VLN is trained primarily for atomic navigation behaviors rather than global exploration, we pair it with a separate VLM [44] that plans where to search and updates the plan using newly observed evidence. On HM3D-v2 and HM3D-OVON, this combination of global search reasoning and learned local navigation achieves state-of-the-art results compared with published methods from [17; 45]. Performance remains consistent across seen, synonymous, and unseen OVON object categories, indicating that the system can handle variations in both object descriptions and the routes required to find them.

4.3. Complementary Agent and Policy Capabilities

Table 3 studies how coding agents and ASENA-VLN contribute to navigation performance. We compare published zero-shot methods, standalone ASENA-VLN, and ASENA instantiated with two coding backends. Our R2R and RxR *Agentic Splits* each contain 100 tasks, following the reduced-set evaluation practice of Uni-LaViRA and Open-Nav [18; 34]. All policy and agent variants use the same task IDs across ten scenes in each benchmark, and each coding agent begins with a fresh workspace. Claude Sonnet 5 runs at high effort, while GPT-6 Astra

Table 4: Embodied question answering. Accuracy (%) and mean normalized steps with differing coverage.

Method	HM-EQA		MT-HM3D	
	Acc.↑	Steps↓	Acc.↑	Steps↓
Explore-EQA [19]	58.4	.52	35.1	.64
3D-Mem [20]	50.4	.63	–	–
Fine-EQA [46]	56.0	.54	–	–
GraphEQA [21]	63.5	.20	45.6	.45
MemoryEQA (Qwen2VL-7B) [22]	–	–	51.2	.40
MemoryEQA (GPT-4o) [22]	63.4	.40	55.1	.41
FAST-EQA [23]	69.2	.65	50.5	.52
Qwen-RobotNav [17]	76.7	.15	54.4	.19
ASENA base (low effort)	70.9	.15	64.8	<u>.10</u>
ASENA hand-designed (efficient)	72.2	.11	63.8	.11
ASENA hand-designed (quality)	<u>78.8</u>	.22	67.1	.24
ASENA self-evolved 17 (low)	74.8	<u>.13</u>	64.9	.08
ASENA self-evolved 17 (high)	81.2	.25	<u>65.5</u>	.22

Table 5: Frozen EQA workspaces: accuracy (%) and normalized exploration steps. Shading marks the final workspace.

Pass	Effort	HM-EQA		MT-HM3D	
		Acc.↑	Steps↓	Acc.↑	Steps↓
Base	Low	70.9	.153	64.8	.101
4	Low	73.4	<u>.136</u>	66.5	<u>.091</u>
9	Low	<u>75.8</u>	.147	<u>65.5</u>	.116
11	Low	75.4	.152	65.3	.114
17	Low	74.8	.127	64.9	.076
17	High	81.2	.253	<u>65.5</u>	.224

runs at maximum effort. Four RxR episodes produce non-finite NE or SPL values; we count them as failures for SR and compute NE and SPL over the remaining 96 episodes.

Strong coding agents navigate effectively without a learned policy. Using only geometric and programmatic tools, GPT-6 Astra achieves state-of-the-art performance and surpasses standalone ASENA-VLN. On R2R, its success rate also approaches the reported human performance of 90% SR. Its strong reasoning allows it to interpret instructions, construct routes, monitor progress, and recover from errors without relying on a learned navigation policy. These results establish coding agents themselves as strong zero-shot navigation systems.

Learned navigation improves weaker coding backends. The benefit of ASENA-VLN is more pronounced for Claude Sonnet 5. Adding the policy improves both success and path efficiency, increasing SR by 11 percentage points on both R2R and RxR. The specialized policy therefore compensates for limitations in route execution and enables a weaker coding backend to complete more tasks. In comparison, GPT-6 Astra already performs strongly without the policy and benefits less from it; on RxR, enabling the policy slightly reduces its success rate. The value of learned navigation therefore depends on the coding backend and how effectively it incorporates policy predictions into its plans.

Learned navigation reduces repeated agent interaction. A single ASENA-VLN call executes several motion decisions, reducing repeated observation and action requests. Mean tool calls fall for every backend and benchmark (Table 3): by 46.9%/51.0% on R2R/RxR for GPT-6 Astra and 59.2%/62.5% for Claude Sonnet 5. The policy therefore absorbs low-level navigation even when success changes little. These call reductions show that learned navigation can reduce agent interaction while improving less capable backends, complementing strong agents’ direct programmatic control.

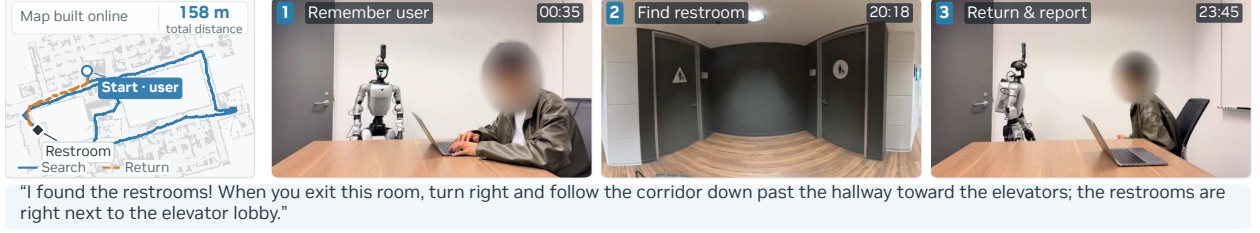
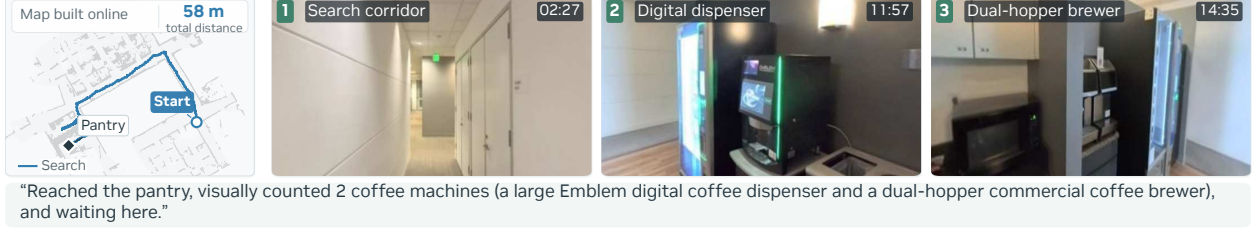
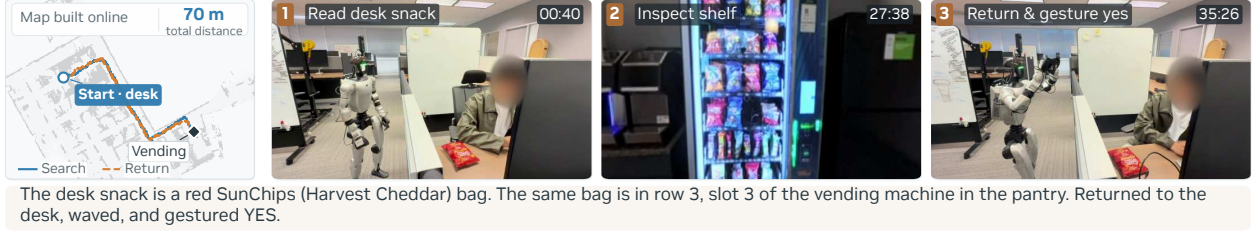
A “Could you find the restroom and then come back and tell me where it is?”**B “Go to the pantry and count the coffee machines. Wait there.”****C “I’m in a meeting, so don’t speak. Find the vending machine and check if it has the snack on my desk. Come back and answer with body language.”**

Figure 5: Real-world navigation and interaction on the G1 robot. Three supervised tasks pair online maps and recorded trajectories with observations and responses. Blue and dashed orange traces show search and return. Distances are estimated from drive odometry. Prompts and responses are condensed for display.

4.4. Retained Experience Improves Recurring Tasks

We evaluate workspace evolution by repeatedly presenting the same tasks, scenes, instructions, and initial poses. In each pass, sixteen Claude Sonnet 5 workers at high effort execute the tasks and may retain programs, notes, and prior action sequences in private workspaces. Between passes, an improver uses their traces and feedback to update the canonical workspace. We run campaigns with and without ASENA-VLN on R2R and RxR using ground-truth feedback, while keeping all model, policy, and controller weights fixed.

As shown in Fig. 4, success improves across all four campaigns. The variants with and without ASENA-VLN reach similar final success rates on R2R, while the policy retains a clearer advantage on the longer RxR instructions. Tool-equipped workspaces also maintain higher SPL on both benchmarks, showing that ASENA-VLN improves path efficiency even when final success rates converge. Meanwhile, logged worker and improver costs decrease as the workspace evolves. These results show that retained experience can improve recurring-task performance and reduce cost without updating model weights, while learned navigation continues to provide an efficiency advantage.

4.5. Embodied Question Answering

Embodied question answering (EQA) requires an agent to explore an environment, gather relevant visual evidence, and answer a question using as little motion as possible. We evaluate ASENA on HM-EQA and MT-HM3D using their released initial poses [19; 22]. Following the Explore-EQA teleportation protocol, each episode has a nominal step budget of $3\sqrt{A}$ for a scene of area A . We report answer accuracy and normalized exploration steps, defined as the fraction of this nominal budget consumed by the agent.

Hand-designed reference configurations. Our EQA agent composes navigation, visual inspection, object search, place memory, and answer-generation tools. We first construct two hand-designed configurations as reference points. The *quality* configuration prioritizes collecting sufficient evidence to answer correctly and receives no explicit instruction to minimize exploration. The *efficient* configuration is prompted to treat exploration steps as a limited resource and gather evidence more selectively. On HM-EQA, we give the efficient agent a target budget equal to 30% of the nominal allowance, while retaining the same simulator hard limit for both configurations. These configurations provide manually designed operating points for accuracy and efficiency rather than a controlled ablation of a single component.

Workspace-evolution protocol. We next study whether the agent can improve this tradeoff by revising its workspace. We construct fixed evolution pools containing 398 HM-EQA questions and 1,183 MT-HM3D questions from training scenes. During each pass, sixteen workers sample 50 questions from each benchmark, execute them using private workspaces, and reflect on their outcomes. A separate improver aggregates their traces and edits to update the canonical workspace, while all model weights remain fixed. Questions not sampled by the workers are used for monitoring, and their aggregate results are also provided to the improver.

Workspace evolution improves the cost–accuracy tradeoff. With workspace frozen, pass 17 achieves the highest HM-EQA accuracy (81.2%), while pass 4 performs best on MT-HM3D (66.5%) (Table 5). At low reasoning effort, evolved workspaces use steps comparable to the hand-designed efficient configuration while achieving higher accuracy (Table 4). Improvements vary across revisions as the workspace acquires new routines and refines existing guidance, resulting in different trade-offs between accuracy and exploration efficiency.

Learned navigation reduces exploration. In a paired ablation on 132 MT-HM3D questions, ASENA-VLN reduces median normalized exploration steps by approximately 19% with little change in accuracy. Consistent with our previous R2R and RxR results, learned navigation improves efficiency by reducing repeated agent interaction.

4.6. Real-World Applications

We evaluate ASENA on three supervised missions using a Unitree G1 (Fig. 5). In each mission, the robot searches for targets that are not initially visible in a previously unseen environment without a pre-built map. It gathers visual and geometric evidence online and uses memory and multimodal interaction to complete the user’s request.

In the restroom mission, the robot locates doors near the elevator lobby, reasons over its online map to find a shorter route from the user, and returns with directions. In the pantry mission, it finds the room, inspects a dispenser and brewer, and reports two coffee machines. The snack-comparison mission requires the robot to inspect a reference snack, find a vending machine, determine whether the same item is available, return to the user, and generate a whole-body gesture to answer. These missions demonstrate open-world search, evidence collection, spatial reasoning, memory, and multimodal interaction. Together, these missions illustrate how online program revision extends navigation into adaptive search and user interaction beyond a predefined skill repertoire.

Fig. 1 shows condensed recorded code that turns LiDAR geometry into a map and proposed waypoints (orange). After safety checks and operator approval, execution returns a camera observation for further reasoning. Generated gestures pass online simulation checks before SONIC execution.

5. Discussion and Conclusion

ASENA unifies coding-agent programming, supervised execution, and persistent skills, improving recurring-task performance with fixed model weights. Our optional 4B monocular ASENA-VLN policy, trained on curated

atomic-navigation data, achieves leading R2R/RxR success rates with fewer agent interactions. ASENA also achieves leading EQA accuracy. On G1, online programming extends navigation to search, inspection, and interaction, including whole-body responses synthesized and validated online.

Real-world tasks take 10–20 minutes, motivating faster reasoning and execution. Future work will strengthen safeguards for dynamic environments and extend workspace evolution to learned policy updates.

Acknowledgment

We thank Jarred Travers, Amanpreet Singh, Peter Pham, Jinhyung (David) Park, Xiangchen Tian, and Tingwu Wang for their help with real robot infrastructure.

References

- [1] P. Anderson, Q. Wu, D. Teney *et al.*, “Vision-and-language navigation: Interpreting visually-grounded navigation instructions in real environments,” in *CVPR*, 2018. [2](#), [6](#), [14](#)
- [2] A. Ku, P. Anderson, R. Patel, E. Ie, and J. Baldridge, “Room-across-room: Multilingual vision-and-language navigation with dense spatiotemporal grounding,” in *EMNLP*, 2020. [6](#), [14](#)
- [3] J. Zhang, K. Wang, R. Xu *et al.*, “NaVid: Video-based VLM plans the next step for vision-and-language navigation,” in *RSS*, 2024. [3](#), [5](#)
- [4] A.-C. Cheng, Y. Ji, Z. Yang *et al.*, “NaVILA: Legged robot vision-language-action model for navigation,” in *RSS*, 2025. [5](#)
- [5] M. Wei, C. Wan, X. Yu *et al.*, “StreamVLN: Streaming vision-and-language navigation via slowfast context modeling,” in *ICRA*, 2026. [2](#), [3](#), [5](#)
- [6] J. Liang, W. Huang, F. Xia *et al.*, “Code as policies: Language model programs for embodied control,” in *ICRA*, 2023, pp. 9493–9500. [2](#)
- [7] L. Fu, J. Yu, K. El-Refai *et al.*, “CaP-X: A framework for benchmarking and improving coding agents for robot manipulation,” *arXiv preprint arXiv:2603.22435*, 2026. [2](#)
- [8] R. Lu, Y. Wu, E. Kou *et al.*, “ASPIRE: Agentic /skills discovery for robotics,” *arXiv preprint arXiv:2607.00272*, 2026. [2](#)
- [9] G. Wang, Y. Xie, Y. Jiang *et al.*, “Voyager: An open-ended embodied agent with large language models,” *Transactions on Machine Learning Research*, 2024. [2](#)
- [10] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *NeurIPS*, 2023. [2](#)
- [11] W. Xiao, J. Xie, T. Zhang *et al.*, “ENPIRE: Agentic robot policy self-improvement in the real world,” *arXiv preprint arXiv:2606.19980*, 2026. [2](#)
- [12] S. Chen, P.-L. Guhur, M. Tapaswi, C. Schmid, and I. Laptev, “Think global, act local: Dual-scale graph transformer for vision-and-language navigation,” in *CVPR*, 2022. [3](#)
- [13] D. An, H. Wang, W. Wang *et al.*, “ETPNav: Evolving topological planning for vision-language navigation in continuous environments,” *TPAMI*, vol. 47, no. 7, pp. 5130–5145, 2025. [3](#)
- [14] D. Shah, B. Osiński, B. Ichter, and S. Levine, “LM-Nav: Robotic navigation with large pre-trained models of language, vision, and action,” in *CoRL*, 2023. [3](#)

- [15] Y. Long, W. Cai, H. Wang, G. Zhan, and H. Dong, “InstructNav: Zero-shot system for generic instruction navigation in unexplored environment,” in *CoRL*, 2025. 3, 6
- [16] M. Wei, C. Wan, J. Peng *et al.*, “Ground slow, move fast: A dual-system foundation model for generalizable vision-language navigation,” in *ICLR*, 2026. 3, 5
- [17] Qwen Team, “Qwen-RobotNav technical report: A scalable navigation model designed for an agentic navigation system,” 2026, arXiv:2606.18112. 3, 5, 7, 8
- [18] H. Ding, S. Zhang, Z. Xu *et al.*, “Uni-LaViRA: Language-vision-robot actions translation for unified embodied navigation,” *arXiv preprint arXiv:2605.27582*, 2026. 3, 6, 7
- [19] A. Z. Ren, J. Clark, A. Dixit, M. Itkina, A. Majumdar, and D. Sadigh, “Explore until confident: Efficient exploration for embodied question answering,” in *RSS*, 2024. 3, 8, 9
- [20] Y. Yang, H. Yang, J. Zhou *et al.*, “3D-Mem: 3D scene memory for embodied exploration and reasoning,” in *CVPR*, 2025. 3, 8
- [21] S. Saxena, B. Buchanan, C. Paxton *et al.*, “GraphEQA: Using 3D semantic scene graphs for real-time embodied question answering,” in *CoRL*, 2025. 3, 8
- [22] M. Zhai, Z. Gao, Y. Wu, and Y. Jia, “Memory-centric embodied question answering,” 2025, arXiv:2505.13948. 3, 8, 9
- [23] H. Zhang, N. Savaliya, F. Siddiqui, and E. Sachdeva, “FAST-EQA: Efficient embodied question answering with global and local region relevancy,” in *WACV*, 2026. 3, 8
- [24] Foxglove Developers, “MCAP: Serialization-agnostic log container file format,” <https://github.com/foxglove/mcap>, 2024, accessed: September 15, 2026. 3, 5
- [25] Z. Luo, Y. Yuan, T. Wang *et al.*, “SONIC: Supersizing motion tracking for natural humanoid whole-body control,” *Science Robotics*, vol. 11, no. 117, p. eaed4592, 2026. 4
- [26] J. Zhang, K. Wang, S. Wang *et al.*, “Uni-NaVid: A video-based vision-language-action model for unifying embodied navigation tasks,” in *RSS*, 2025. 5
- [27] W. Zhu, Z. Zhang, X. Wang *et al.*, “NaVIDA: Vision-language navigation with inverse dynamics augmentation,” *arXiv preprint arXiv:2601.18188*, 2026. 5
- [28] S. Zeng, D. Qi, X. Chang *et al.*, “JanusVLN: Decoupling semantics and spatiality with dual implicit memory for vision-language navigation,” in *ICLR*, 2026. 5
- [29] Z. Xin, W. Li, Y. Jiang *et al.*, “DecoVLN: Decoupling observation, reasoning, and correction for vision-and-language navigation,” in *CVPR*, 2026. 5
- [30] Z. Zhang, W. Zhu, H. Pan *et al.*, “ActiveVLN: Towards active exploration via multi-turn RL in vision-and-language navigation,” in *ICRA*, 2026. 5
- [31] Q. Wang, M. Li, J. Guan *et al.*, “Qwen-VLA: Unifying vision-language-action modeling across tasks, environments, and robot embodiments,” *arXiv preprint arXiv:2605.30280*, 2026. 5
- [32] Intern Robotics, “InternVLA-N1: An open dual-system vision-language navigation foundation model with learned latent plans,” 2025, technical report, Shanghai AI Laboratory. 5
- [33] R. Dang, J. Guo, B. Hou *et al.*, “RynnBrain: Open embodied foundation models,” *arXiv preprint arXiv:2602.14979*, 2026. 5

- [34] Y. Qiao, W. Lyu, H. Wang *et al.*, “Open-Nav: Exploring zero-shot vision-and-language navigation in continuous environment with open-source LLMs,” in *ICRA*, 2025. 6, 7
- [35] K. Chen, D. An, Y. Huang *et al.*, “Constraint-aware zero-shot vision-language navigation in continuous environments,” *TPAMI*, vol. 47, no. 11, pp. 10 441–10 456, 2025. 6
- [36] H. Yin, H. Wei, X. Xu, W. Guo, J. Zhou, and J. Lu, “GC-VLN: Instruction as graph constraints for training-free vision-and-language navigation,” in *CoRL*, 2025. 6
- [37] W. Zheng, Y. Ge, and L. Itti, “Three-Step Nav: A hierarchical global-local planner for zero-shot vision-and-language navigation,” in *AISTATS*, 2026. 6
- [38] J. Zhou, X. Zhao, G. Zhou *et al.*, “Embodied agents take control: Minimal-interface zero-shot agents rival industrial-scale policies in vision-and-language navigation,” *arXiv preprint arXiv:2607.26148*, 2026. 6
- [39] S. Bai, Y. Cai, R. Chen *et al.*, “Qwen3-VL technical report,” 2025, arXiv:2511.21631. 6
- [40] Z. Wang, J. Li, Y. Hong *et al.*, “Scaling data generation in vision-and-language navigation,” in *ICCV*, 2023. 6, 14
- [41] Z. Wang, J. Li, Y. Hong *et al.*, “Bootstrapping language-guided navigation learning with self-refining data flywheel,” in *ICLR*, 2025. 6, 14
- [42] OpenAI, “GPT-5.5,” 2026. 6
- [43] Z. Cai, R. Wang, C. Gu *et al.*, “Scaling spatial intelligence with multimodal foundation models,” in *CVPR*, 2026. 7, 14
- [44] Google DeepMind, “Gemini 3.1 Flash-Lite model card,” Google DeepMind, Tech. Rep., 2026. 7
- [45] H. Zhu, Z. Li, Z. Liu *et al.*, “SysNav: Multi-level systematic cooperation enables real-world, cross-embodiment object navigation,” *arXiv preprint arXiv:2603.06914*, 2026. 7
- [46] K. Jiang, Y. Liu, W. Chen *et al.*, “Beyond the destination: A novel benchmark for exploration-aware embodied question answering,” in *ICCV*, 2025. 8
- [47] Y. Hong, C. Rodriguez, Q. Wu, and S. Gould, “Sub-instruction aware vision-and-language navigation,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 2020, pp. 3360–3376. 14
- [48] S. K. Ramakrishnan, A. Gokaslan, E. Wijmans *et al.*, “Habitat-matterport 3D dataset (HM3D): 1000 large-scale 3D environments for embodied AI,” in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, vol. 1, 2021. 14
- [49] Insta360, “X5: Webcam live streaming tutorial using OBS,” <https://onlinemanual.insta360.com/x5/en-us/camera/appuse/obs>, accessed September 9, 2026. 18
- [50] Livox, “MID-360 specifications,” <https://www.livoxtech.com/mid-360/specs>, accessed September 9, 2026. 18

A. Supplementary Material

A.1. Atomic Instruction Format and Training Data

Stored field	Type	Meaning
goal_type goal_value	category; text	Goal category and text. Recaptioned records can store just the destination phrase, while raw instructions and generated atoms retain the complete instruction. Categories include object, area, ego, region and point.
route_steps	ordered list	Ordered directions and landmark cues, when separately annotated.
constraints	list	Avoidance, traversal, or relative-position requirements.
end_pose	anchor list	Target, spatial relation, and distance derived from scene geometry, relative to an object, a room, or the robot’s starting position.
quality	1–5	Quality annotation or assigned setting used to condition the policy. R2R/RxR evaluation uses 5.

Table 6: Stored navigation annotations. The policy receives goal text, quality, and optional end-pose anchors (Tab. 10). Coordinate-based point goals use a separate input format.

Dataset	Supervision	Weighted pool (M)	Environments
R2R [1]	traj, pixel, CoT	1.502	MP3D
R2R (augmented)	traj, pixel	3.290	MP3D
FGR2R [47]	traj, pixel	0.704	MP3D
RxR [2]	traj, pixel, CoT	4.191	MP3D
RxR (augmented)	traj, pixel	4.836	MP3D
ScaleVLN [40]	traj, pixel, CoT	3.120	HM3D
SRDF [41]	traj, pixel	3.916	HM3D
ObjectNav MP3D v3	traj, pixel	0.707	MP3D
ObjectNav HM3D v3 [48]	traj, pixel	2.888	HM3D
Nav-atoms (synthetic, MP3D)	traj, pixel	1.443	MP3D
Nav-atoms (synthetic, HM3D)	traj, pixel	1.825	HM3D
VLN-MME + other general VQA	VQA	0.964	various
SenseNova-SI [43]	VQA	8.164	various
Trajectory pool		28.422	
VQA pool		9.128	

Table 7: Training pools for ASENA-VLN. Pool sizes account for quality and backward-action filtering, alternative views, chain-of-thought (CoT) variants, repeated trajectory-ending windows, and sampling weights. Counts are rounded independently and include repeated examples. They describe the available pools, not the number of samples consumed during training. Sampling details appear in Sec. A.1.

Training Sampling. Training uses 64 devices and an effective batch size of 512. Each device processes three trajectory examples and one VQA example per step, with gradients accumulated over two steps. At step 8,400, training has drawn 4,300,800 samples, including repeats, and reached epoch 0.1178. The VQA pool sets the epoch length to 71,309 updates. The larger trajectory pool is therefore only partly sampled in each epoch. To emphasize stopping behavior, we add five extra copies of each eligible trajectory-ending window. The pools also include alternative views, CoT variants, and repeated observations.

A.2. Jetson Thor Quantization and Runtime

To run the policy on the G1’s Jetson AGX Thor, we apply post-training FP8 quantization with NVIDIA ModelOpt and build inference engines with TensorRT-Edge-LLM. These measurements use an earlier ASENA-VLN checkpoint at step 8,400, separate from the benchmark policy in Tab. 1. Every FP8 variant quantizes the language model. Variants labeled “BF16 vision” keep the 415M-parameter vision encoder in BF16. The embedding and output projection remain unquantized in all variants.

Navigation-only engines need to predict motion tokens rather than general text. We therefore reduce the output vocabulary from 152,596 to 768 entries while retaining all 660 trajectory tokens. This removes 388.7M parameters from the output projection evaluated at each decoding step.

Variant	Export size (GiB)	Visual	Prefill	Decode (32) Latency (ms)	Total	Speedup
PyTorch BF16	9.01	167.3	363.4	2,275.5	2,806	1.00×
FP8, BF16 vision	5.63	73.3	128.7	847.4	1,050	2.67×
FP8, BF16 vision, reduced vocab	4.91	78.3	125.5	694.1	898	3.13×
FP8, reduced vocab	4.53	64.9	129.1	683.4	877	3.20×

Table 8: Policy inference on the Unitree G1’s Jetson AGX Thor. These measurements use an earlier checkpoint, separate from the benchmark policy. All variants use batch size 1, the same eight-frame mixed-resolution history, and 32 greedy decoding steps. Totals sum the three components. Speedups are computed from unrounded sums. Measured BF16 end-to-end latency is 2,735 ms. Export size describes the corresponding ONNX files. The BF16 model occupies 8.28 GiB of allocated memory after loading. This is not a peak runtime measurement. BF16 values are warm medians of five runs: prefill is time to first token minus visual encoding, and decode scales the remaining 31 tokens to 32 steps. TensorRT-Edge-LLM visual and prefill values are 10-run means. Decode is the median of five runs.

Variant	First-waypoint agreement	Exact-trajectory agreement
FP8	90.23%	75.39%
FP8, BF16 vision	91.41%	82.81%

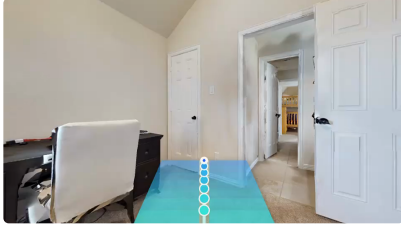
Table 9: Prediction agreement between the earlier quantized model and its BF16 version on 256 held-out inputs. First-waypoint agreement requires the first predicted (x, y, θ) tuple to match. Exact-trajectory agreement requires the entire predicted trajectory to match. These metrics measure output consistency, not navigation success.

Tab. 8 shows where the inference time is spent. FP8 with the reduced vocabulary lowers the sum of component latencies from 2,806 to 877 ms, a 3.20× speedup over PyTorch BF16 under the same input settings. Decoding remains the largest cost at 683.4 ms, or 78% of the FP8 total. Its cost can be reduced without changing visual encoding or prefill: among the BF16-vision variants, reducing the vocabulary cuts decoding from 847.4 to 694.1 ms (18.1%), while the other two stages stay nearly unchanged.

Tab. 9 checks how closely quantized predictions match the BF16 model on held-out inputs. Keeping the vision encoder in BF16 improves agreement for both the first waypoint and the full trajectory. These checks measure output consistency. Navigation success requires separate rollout evaluation. On-device checks also confirmed valid navigation-token IDs and correctly formatted eight-waypoint trajectories.

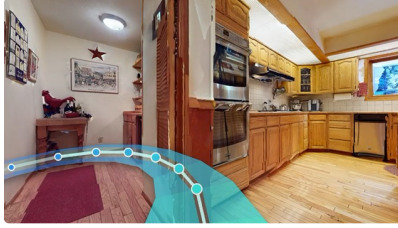
A.3. Atomic Navigation Dataset Samples

Metric motion



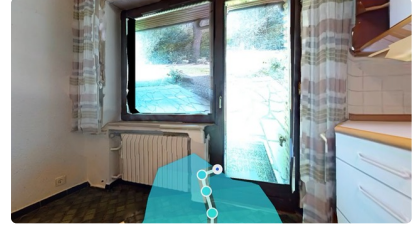
Stop after you take 4 steps forward.

Pure rotation



You will need to turn all the way around to face behind you.

Heading-relative goal



Walk over to the built-in dishwasher off to your left.

Fine alignment



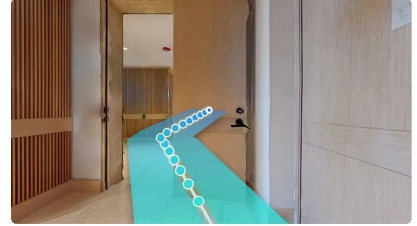
Sidestep right toward the bathroom exhaust fan.

Room goal



Stop there once you find a bedroom.

Door traversal



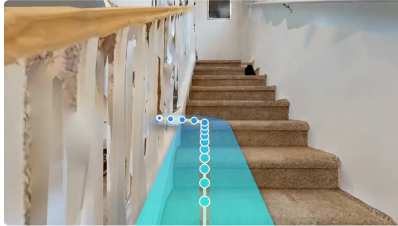
Step into the kitchen through the door.

Corridor following



Head down the corridor and stop at the end.

Floor change



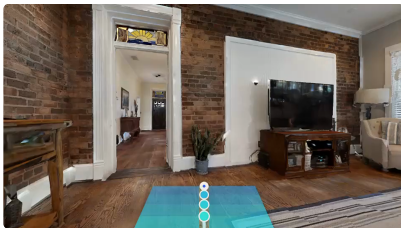
Head upstairs.

Heading-relative goal



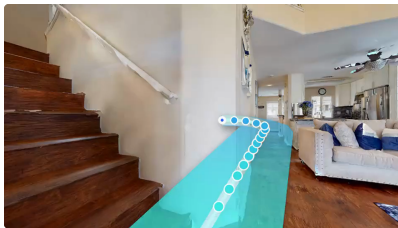
Turn around and walk to the white wall-mounted cabinet right behind you.

Metric motion



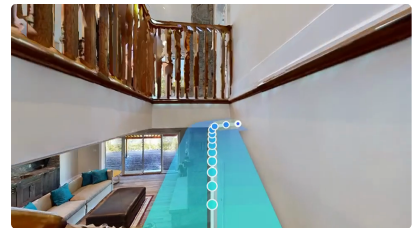
Can you move forward 0.5 meters?

Door traversal



Step into the hallway.

Floor change



You should head down the stairs.

Figure 6: Atomic navigation training examples. Each observation is paired with an instruction and an overlaid trajectory. The overlays do not use metric camera projection. The examples span metric motion, rotation, relative goals, alignment, room goals, doors, corridors, and floor changes.

A.4. Navigation Instructions and Policy Inputs

Tab. 10 shows how navigation instructions are represented as policy inputs. Each input contains the goal text and a quality setting. An optional end-pose anchor describes where the robot should finish relative to an object, a room, or its starting position.

Table 10: Navigation instructions and policy inputs. Each JSON input contains the goal text and a quality setting, with an optional end-pose anchor that specifies where the robot should finish.

Navigation instruction	Policy input (JSON)
Route following Descend down the stairs. Turn left and stop in the room.	<pre>{ "goal": "Descend down the stairs. Turn left and stop in the room. ", "quality": 5 }</pre>
Metric motion Could you move forward 1 meters for me?	<pre>{ "goal": "Could you move forward 1 meters for me?", "quality": 5, "end_pose": [{ "target": "agent_start", "relation": "forward", "distance_m": 1.0 }] }</pre>
Relative object goal Walk over to the built-in dishwasher off to your left.	<pre>{ "goal": "Walk over to the built-in dishwasher off to your left.", "quality": 5, "end_pose": [{ "target": "built-in dishwasher", "relation": "near", "distance_m": 0.55 }] }</pre>
Room goal Stop there once you find a bedroom.	<pre>{ "goal": "Stop there once you find a bedroom.", "quality": 5, "end_pose": [{ "target": "bedroom", "relation": "inside", "distance_m": 0.0 }] }</pre>
Floor change Make your way downstairs to the hallway.	<pre>{ "goal": "Make your way downstairs to the hallway.", "quality": 5, "end_pose": [{ "target": "hallway", "relation": "inside", "distance_m": 0.0 }] }</pre>

Directions, route cues, and constraints stay in the instruction text. End-pose distances are expressed in meters and rounded to two decimal places. A room goal uses the relation `inside` with distance 0.0. All five examples use quality 5. The policy receives this instruction input together with its visual history.

A.5. G1 Hardware and Navigation Settings

The G1 carries a mast-mounted panoramic camera and an inverted Livox Mid-360 LiDAR (Fig. 7). A Jetson AGX Thor runs the local sensor and motion services. The camera interface supports both the Insta360 X5 and Ricoh Theta X. Tab. 11 lists the sensor and state outputs, and Tab. 12 summarizes the navigation settings and command limits.

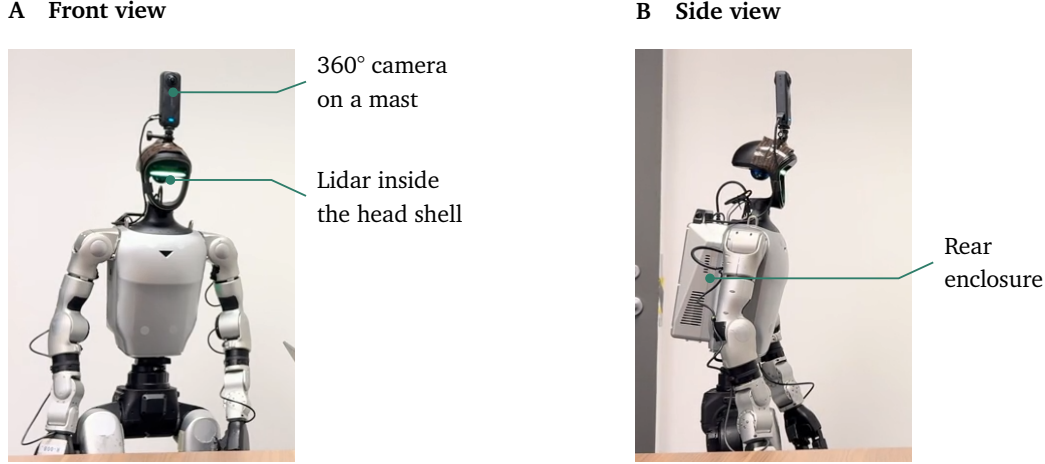


Figure 7: G1 sensor mounting. Front and side views showing sensor placement on the robot.

Table 11: Sensor and state interfaces. Nominal specifications and configured publication rates. Lidar angles use the manufacturer’s orientation.

Component	Native input / specification	Configured output
Insta360 X5	USB panorama: 2880×1440 ; 30 fps [49]	6 Hz JPEG; 180° yaw correction
Livox Mid-360	200,000 points/s; field of view: 360° H, -7° to 52° V [50]	SLAM bridge: 10 Hz; at most 20,000 points/batch
Body state	29 joint positions/velocities; IMU, odometry and mode	20 Hz

Table 12: Navigation settings and safeguards. The slow-walk controller limits the speed, duration, and size of individual commands and path segments.

Parameter	Value	Parameter	Value
Command publication	10 Hz	Path lookahead	0.40 m
Translation speed bounds	0.12–0.35 m/s	Braking lead time	0.60 s
Near-goal speed (≤ 0.5 m)	0.25 m/s	Velocity-command expiry	2.0 s
Arrival tolerances	0.10 m; 4°	Active-job heartbeat timeout	3.0 s
Move / turn command limits	2.0 m; 180°	Path chunk limits	6.0 m; 16 points
Move-job timeout	25 s	Path-job timeout	45 s

The slow-walk settings control motion speed, stopping tolerances, command size, and timeouts. They can be adjusted at runtime, and a separate walking gait is also available. Camera yaw correction sets the panorama’s horizontal orientation. The robot’s head and body partially occlude the inverted LiDAR’s view.